# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 3 – ARCHITECTURE / MODELING (DEVSECOPS METHODOLOGY)

| | | | |
|---|---|---|---|
| **Course Code:** | CSA5802 | **Course Title:** | DEVSECOPS ENGINEERING AND APPLICATION SECURITY |
| **CO Assessed:** | CO4 – Precision (BL1, BL2, BL3) | **Assessment:** | Assessment Tool 3 |
| **Weightage:** | 30% of CIA | **Total Marks:** | 100 |
| **CO4 Statement:** | Develop Complete DevSecOps Architecture, Methodology Documentation and GitHub Submission. | | |
| **Student Name:** | Krithika M | **Reg. No.:** | 192521385 |
| **Section / Batch:** | B-Tech IT | **Date:** | 19 08 2026 |

Instructions to Students

- Develop a complete and original DevSecOps architecture showing the end-to-end secure software delivery workflow.
- Include development, source control, CI/CD, automated security testing, artifact management, deployment, monitoring and feedback stages.
- Document the methodology clearly, including security controls, tools, workflow relationships and key design decisions.
- Use clear labels, consistent symbols, proper alignment and professionally formatted architecture diagrams.
- Submit the editable architecture source files, methodology documentation and an accessible GitHub repository containing all required artifacts.

| Question Type | Focus Area | Questions | Marks |
|---|---|---|---|
| ARCHITECTURE / MODELING | DevSecOps Methodology | Q1 | Q1 – 100 |
| **GRAND TOTAL:** | | | **100 Marks** |

## RUBRICS FOR CALCULATION

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Complete DevSecOps Architecture Design | 20% | Develops a comprehensive and technically accurate DevSecOps architecture integrating development, security, CI/CD, infrastructure and operations components | Develops a clear DevSecOps architecture containing most required components with only minor technical or integration gaps | Develops a basic DevSecOps architecture with limited integration or several missing components | Provides an incomplete or technically inaccurate DevSecOps architecture with weak component integration |
| Security Integration Across DevSecOps Lifecycle | 30% | Effectively integrates security controls and automated security testing across planning, coding, build, test, deployment and operations stages | Integrates appropriate security controls across most DevSecOps stages with only minor omissions | Shows basic security integration but several lifecycle stages or security controls are missing | Shows limited or incorrect security integration across the DevSecOps lifecycle |
| Methodology and Workflow Documentation | 20% | Clearly documents the complete DevSecOps methodology, workflow, tools, responsibilities and security checkpoints with strong technical justification | Documents the DevSecOps methodology and workflow clearly with minor gaps in explanation or justification | Provides basic methodology documentation with limited workflow details or technical explanation | Provides incomplete or unclear methodology documentation with weak technical reasoning |
| GitHub Repository and Submission Quality | 20% | Submits a complete, well-organized GitHub repository containing architecture artifacts, documentation and required files with clear structure and version history | Submits a properly organized GitHub repository containing most required artifacts with minor structural or documentation issues | Submits a partially organized repository with missing artifacts, limited documentation or inconsistent file structure | GitHub submission is incomplete, poorly organized or missing essential architecture and methodology artifacts |
| Technical Clarity and Professional Presentation | 10% | Presents the DevSecOps architecture, methodology and repository professionally with clear relationships, consistent terminology and strong technical clarity | Presents the work clearly and professionally with only minor clarity or formatting issues | Presents the work with basic clarity but noticeable inconsistencies or limited technical explanation | Presents the work poorly with unclear relationships, inconsistent terminology or insufficient technical explanation |

## Rubrics:

| Criteria | Weightage | Q5 (10) | Total Marks (50) |
|---|---|---|---|
| Complete DevSecOps Architecture Design | 25% | | |
| Security Integration Across DevSecOps Lifecycle | 30% | | |
| Methodology and Workflow Documentation | 20% | | |
| GitHub Repository and Submission Quality | 15% | | |
| Technical Clarity and Professional Presentation | 10% | | |
| **Total per Question** | **100 Marks** | | |

100

**Section 1: System Overview and Strategic Architecture**

1.1 Executive Summary

Modern software engineering demands high-velocity delivery without sacrificing security. Modern AI-driven coding assistants accelerate software production, but legacy application security testing tools struggle under high false-positive rates, alert fatigue, and delayed feedback loops.

SecureCode AI Sentinel addresses this gap as a DevSecOps platform. It embeds Artificial Intelligence (Machine Learning, Large Language Models, and Static Analysis ASTs) directly into every stage of the Software Development Life Cycle (SDLC).

By shifting security controls to the left (Developer IDE / Code Submission) while maintaining continuous monitoring to the right (Runtime / Production), the platform transforms security from a manual bottleneck into an automated, continuous guardrail.

1.2 Core Architectural Objectives

- Shift-Left Security with AI Assistance: Detect vulnerabilities directly inside the developer's Integrated Development Environment (IDE) and Pull Request (PR) workflow using real-time ML-driven pattern recognition.
- Context-Aware Vulnerability Prioritization: Reduce false positives by applying Large Language Models (LLMs) and reachability analysis to verify whether a vulnerable function in a dependency is actually executable in the application context.
- Automated Remediation (Auto-Fixing): Generate automated pull requests containing contextually verified code patches to reduce Mean Time to Remediate (MTTR).
- Zero-Trust Supply Chain Security: Enforce cryptographic artifact signing, Software Bill of Materials (SBOM) generation, and automated license compliance at build time.
- Continuous Policy-as-Code Enforcement: Maintain unified security policies managed centrally and executed automatically across all pipeline stages.

1.3 Key Architectural Principles

| Principle | Technical Implementation |
|---|---|
| 1. Frictionless Developer Experience | Embed feedback inside existing developer interfaces (VS Code, JetBrains, GitHub PRs, GitLab MRs) to eliminate context switching. |
| 2. Defense-in-Depth | Apply multiple security layers: Pre-commit, SAST, SCA, Secrets Scanning, DAST, Container Security, and Runtime Protection (RASP). |
| 3. Contextual AI Triage | Use LLMs trained on Security AST data to filter out non-exploitable vulnerabilities, eliminating low-risk alert noise. |
| 4. Policy as Code | Define security guardrails using Open Policy Agent (OPA) declarative files stored alongside code repositories. |
| 5. Continuous Feedback Loop | Feed production threat insights back into pre-production development policies to prevent repeat vulnerabilities. |

**Section 2: End-to-End DevSecOps Workflow Architecture**

2.1 Workflow Stage Breakdown

- Stage 1: Local Development Workstation
    - Developer Workstation (VS Code / JetBrains / Eclipse)
    - IDE Plugin: Local AI AST Scanner for real-time syntax & vulnerability checks
    - Local Hook: Pre-Commit Git Hook for entropy-based secrets detection & linting
- Stage 2: Source Control & Pre-Merge
    - Source Code Repository (GitHub Enterprise / GitLab / Bitbucket)
    - Webhook Event triggered to invoke the AI Review Bot
    - AI PR Engine: Deep contextual code analysis & semantic diff engine
    - Automated Bot Commenting: Inline security recommendations and auto-remediation PR generation

- Stage 3: CI/CD Pipeline & Automated Security Testing
    - Continuous Integration Engine (GitHub Actions / Jenkins / GitLab CI)
    - Parallel Step A: Deep SAST (Semantic Code Scanning & Taint Analysis)
    - Parallel Step B: SCA & Supply Chain (Dependency Tree & SBOM Generation via CycloneDX)
    - Parallel Step C: Hardened Secrets Scanning (Regex + High Entropy Models)
    - Parallel Step D: IaC Security Scan (Terraform / Helm Misconfigurations)
    - AI Aggregator Engine: Correlation, reachability verification & vulnerability scoring
    - Security Policy Gate: Evaluates findings against OPA Rules (Fail -> Block / Pass -> Continue)
    - Container Build & Image Scan: OS/Package CVE scanning
- Stage 4: Artifact Management
    - Secure Artifact Repository (JFrog Artifactory / AWS ECR / Harbor)
    - Cosign / Notary: Cryptographic artifact & container image signing
    - Metadata Store: Attestation storage for SBOM, scan reports, and build lineage
- Stage 5: Deployment & Infrastructure
    - Deployment Orchestrator (ArgoCD / Flux CD / Kubernetes)
    - Admission Controller: OPA Gatekeeper verifies signatures & SBOM metadata before applying
    - Deployment Targets: Staging & Production Kubernetes Clusters
    - Post-Deploy Automated Step: DAST & Dynamic Pen-Testing Agent (API & Application Scanning)
- Stage 6: Runtime Monitoring & Feedback
    - Production Runtime & Observability (Kubernetes / AWS CloudWatch)
    - Runtime Security: eBPF-based application protection & threat detection (Falco)
    - Logging & SIEM: Central dashboards in Splunk / Datadog / Elastic Security
    - Feedback Loop Engine: Auto-ticket creation in Jira + fine-tuning AI models with production data

2.2 Deep-Dive Phase Analysis

| SDLC Stage | Security Objectives | Integrated Tools | AI / Machine Learning Role |
|---|---|---|---|
| 1. Local Development | Catch vulnerabilities before code leaves the workstation. | SecureCode IDE Plugin, GitLeaks, Pre-commit framework. | Transformer-based models evaluate code syntax in real-time as developers type. |
| 2. Source Control & Pre-Merge | Evaluate pull requests and block security regressions. | GitHub Actions, GitLab Webhooks, SecureCode PR Review Bot. | Contextual LLMs parse pull requests, score risk, and write auto-fix patches. |
| 3. CI/CD Pipeline | Comprehensive static, dependency, secret, and infrastructure verification. | Semgrep, SonarQube, Snyk, Trivy, Checkov, Open Policy Agent (OPA). | Graph Neural Networks (GNN) map taint paths and verify exploit reachability. |
| 4. Artifact Management | Ensure software supply chain integrity and cryptographically sign artifacts. | JFrog Artifactory, Sigstore Cosign, Syft/Grype (SBOM). | Automated verification of cryptographic signatures and binary provenance. |
| 5. Deployment | Enforce zero-trust admission control and dynamic runtime scanning. | ArgoCD, OPA Gatekeeper, Kubernetes, OWASP ZAP (DAST). | Dynamic reinforcement agents perform intelligent API fuzzing on live endpoints. |
| 6. Runtime & Feedback | Detect runtime anomalies and feed context back to development. | Datadog, Falco (eBPF), Splunk, Jira Integration Engine. | Unsupervised anomaly detection flags abnormal app behavior and auto-tickets. |

**Section 3: AI-Powered Core Engines & Security Methodologies**

3.1 AI Architecture & Processing Sequence

1. Input Stage: Raw Vulnerability Findings
    - Collects raw findings directly from SAST, SCA, Secrets Scanning, and IaC tools.
2. Layer 1: Semantic & Graph-Based Taint Analysis
    - Converts source code into Abstract Syntax Trees (AST) and Control Flow Graphs (CFG). Graph Neural Networks (GNNs) map how untrusted user inputs flow through the application to identify vulnerable execution sinks (such as SQL queries or system commands).
3. Layer 2: Dependency Reachability Engine
    - Constructs execution call graphs across the codebase to evaluate 3rd-party library CVEs. It verifies whether the specific vulnerable function inside a flagged package is actually invoked by the application code, down-prioritizing unreachable flaws.
4. Layer 3: Context-Aware LLM Auto-Remediation Engine
    - Evaluates application context, removes false positives, and automatically generates functional code patches and pull requests to fix verified vulnerabilities while adhering to existing repository coding styles.

3.2 Integrated Security Control Framework

Static Application Security Testing (SAST)

- Execution: Scans proprietary code for common security flaws (OWASP Top 10, CWE Top 25).
- Control Mechanism: Uses custom rules and AST pattern matching to identify SQL Injection, Cross-Site Scripting (XSS), Command Injection, and Path Traversal.

Software Composition Analysis (SCA) & SBOM

- Execution: Scans direct and transitive dependencies against vulnerability databases (NVD, GitHub Advisory Database).
- Control Mechanism: Generates CycloneDX and SPDX format SBOMs. Blocks dependencies with high-severity CVEs or non-compliant licenses (e.g., GPL-3.0 in proprietary builds).

Secrets Detection & Prevention

- Execution: Runs continuous pattern scanning across all code commits, configuration files, and build assets.
- Control Mechanism: Combines regular expression pattern matching with high-entropy analysis models to detect leaked API keys, database credentials, AWS secrets, and JWT tokens before code gets pushed.

Infrastructure as Code (IaC) Security

- Execution: Evaluates infrastructure configuration files (Terraform, CloudFormation, Kubernetes Manifests, Helm Charts).

- Control Mechanism: Identifies cloud misconfigurations like open S3 buckets, exposed SSH ports, missing encryption settings, and overly permissive IAM roles.

Dynamic Application Security Testing (DAST)

- Execution: Scans staging environments and running endpoints using automated security tests.
- Control Mechanism: Performs black-box and gray-box testing for issues like broken authentication, improper access controls, and rate-limiting flaws.

**Section 4: Security Policy, Governance, and Automated Guardrails**

4.1 Policy-as-Code Implementation (Open Policy Agent - OPA)

Security rules are defined as declarative OPA Rego policies stored directly in source repositories. This approach ensures consistent enforcement across local development, CI/CD pipelines, and Kubernetes admission controllers.

Operational Policy Rules Engine:

- Critical Vulnerability Gate: Block builds containing verified Critical or High-severity vulnerabilities with a known exploit (EPSS > 0.1).
- Secret Leakage Gate: Halt pipelines immediately if any plaintext credentials, private keys, or tokens are discovered.
- Container Integrity Gate: Reject container images that run as the root user or contain unverified binary packages.
- Supply Chain Signing Gate: Block deployments if container images lack a valid cryptographic signature from Sigstore Cosign.

Policy Rule Logic Summary:

- Default state is set to reject/deny.
- Allow execution only if the count of reachable critical vulnerabilities equals zero AND unencrypted exposed secrets equal zero.
- Classify findings as critical reachable vulnerabilities when severity is CRITICAL and reachability status is REACHABLE.
- Classify findings as exposed secrets when secret detection status is EXPOSED.

4.2 Security Quality Gate Matrix

| SDLC Milestone | Gate Criteria (PASS) | Warning Threshold (WARN) | Hard Block Criteria (FAIL) |
|---|---|---|---|
| IDE / Pre-Commit | Clean scan or low-severity findings only. | Medium-severity issues flagged in local editor. | High/Critical secrets or plain-text credentials detected. |
| Pull Request | Zero Critical/High reachable vulnerabilities. | Medium-severity issues without public exploit paths. | Any High/Critical reachable flaw or unapproved license type. |
| CI/CD Build | Complete SBOM generated and zero critical issues. | High-severity issue with no fix available (requires exception). | Critical vulnerability present without an approved exception. |
| Container Registry | Image signed with valid Cosign key pair. | Base image older than 30 days but no active CVEs. | Unsigned image or running as root user context. |
| Runtime Admission | Signature verified and OPA compliance confirmed. | Non-critical runtime policy warning generated. | Invalid provenance attestation or failed signature check. |

**Section 5: Key Design Decisions, Trade-Offs, and Implementation Plan**

5.1 Architectural Design Decisions and Trade-offs

| Design Decision | Architectural Benefit | Trade-off & Mitigation |
|---|---|---|
| 1. AI Reachability Filtering vs. Standard CVE Scanning | Drastically reduces false positives, allowing developers to focus on actionable risks. | Requires additional processing compute power during CI builds.<br><br>mitigation: Cache graph results and run scans asynchronously. |
| 2. Shift-Left IDE Integration vs. Centralized Security Reviews | Prevents security issues early, reducing remediation costs and back-and-forth reviews. | Risk of IDE slowdown and developer fatigue from too many popups.<br>mitigation: Limit IDE checks to light, fast rule engines. |
| 3. Automated PR Remediation vs. Manual Fix Writing | Speeds up patching (MTTR) by generating context-aware fixes. | AI-generated code could introduce unexpected logic bugs.<br>mitigation: Require human review and CI validation on all AI PRs. |
| 4. Strict Admission Control vs. Permissive Deployments | Guarantees only signed, scanned container images run in production. | Could block urgent production hot-fixes if a pipeline step delays.<br>mitigation: Build a break-glass process for emergency overrides. |

5.2 Implementation Roadmap

| Phase | Duration | Core Deliverables & Focus |
|---|---|---|
| Phase 1: Foundation & Ingestion | Months 1 – 2 | Deploy central security platform infrastructure and connect source control tools (GitHub/GitLab). Implement base SAST, SCA, and Secrets scanning within primary CI/CD pipelines. Define initial security policies and set up baseline reporting dashboards. |
| Phase 2: AI Integration & Advanced Scanning | Months 3 – 4 | Integrate IDE plugins to provide real-time feedback directly to developers. Deploy the AI Reachability Engine to filter out false positives and prioritize findings. Roll out automated Software Bill of Materials (SBOM) generation across all builds. |
| Phase 3: Shift-Right Enforcement & Runtime Monitoring | Months 5 – 6 | Configure Sigstore Cosign for container image signing and provenance tracking. Implement OPA Gatekeeper in Kubernetes clusters to enforce deployment security guardrails. Connect eBPF runtime monitoring (Falco) with SIEM tools to close the feedback loop back to development. |

5.3 Enterprise Risk Mitigation Matrix

| Identified Risk | Impact | Automated Mitigation Strategy |
|---|---|---|
| Hardcoded Credentials / Exposed API Keys | CRITICAL | Pre-commit hooks block pushes containing credentials and trigger automated secret rotation workflows. |
| Vulnerable 3rd-Party Dependencies (Supply Chain) | HIGH | Automated SCA scans verify dependency trees and open auto-remediation PRs for safer versions. |
| Cloud Infrastructure Misconfigurations | HIGH | IaC scanning validates Terraform configurations against CIS Benchmarks before cloud provisioning. |
| Deployment of Tampered or Unsigned Images | CRITICAL | Kubernetes Admission Controllers reject images that lack valid cryptographic signatures. |

**Section 6: Conclusion**

The SecureCode AI Sentinel Platform bridges the gap between fast-paced software delivery and rigorous enterprise security. By integrating AI-powered analysis throughout the entire DevSecOps lifecycle—from local IDE development to runtime production monitoring—it replaces manual security reviews with continuous, automated guardrails.

This unified approach dramatically lowers Mean Time to Remediate (MTTR), reduces false positives, and builds long-term resilience into the software supply chain—delivering secure software at the speed of modern innovation.